Du métier Jacquard au binaire

Première NSI

Représenter les entiers avec deux états, en bases 2, 10 et 16, puis en Python

Représentation des données

Objectifs

À la fin de cette séquence, vous saurez :

- relier une information à deux états à son écriture binaire et expliquer le poids d'une position ;
- convertir un entier positif entre les bases 2, 10 et 16 par une méthode explicite ;
- suivre l'état des variables d'une boucle ou d'une fonction dans un tableau ;
- écrire et convertir des entiers en Python avec `0b`, `0x`, `bin`, `hex` et `int` ;
- évaluer le nombre de bits nécessaire et représenter un entier relatif en complément à 2.

Prérequis

- Écriture positionnelle d'un entier en base 10.
- Puissances de 10 et additions simples.
- Python : variables, `if`, boucles `for` et `while`, fonctions avec `return`.
- Division entière `//` et reste `%`.

Sommaire

Partie	Contenu
Partie 1	Deux états et le poids des positions
Partie 2	Convertir les entiers : méthode, base 16 et Python
Partie 3	Taille d'une écriture binaire
Partie 4	Entiers relatifs et complément à 2

Partie 1 — Deux états et le poids des positions

Le problème

Une manufacture textile veut qu'un métier à tisser reproduise automatiquement un motif complexe, toujours identique, sur des centaines de mètres de tissu. Comment donner cette instruction à une machine ?

Au début du XIXe siècle, Joseph Marie Jacquard répond à ce problème avec une **carte perforée** : à chaque position de la carte, il y a un trou ou pas de trou. Cette information commande le passage des fils, motif après motif.

> **Repère**
>
> Une carte perforée ne code pas directement notre écriture binaire actuelle — mais le principe est le même : **deux états suffisent** à commander une mécanique aussi complexe soit-elle.



Fig. 1 - Une carte perforée représente huit positions binaires : trou pour 1, absence de trou pour 0.

Le bit

trou / pas de trou ⟶ deux états ⟶ **0 / 1**

Un de ces deux états s'appelle un **bit** (contraction de l'anglais *binary digit*). C'est la plus petite unité d'information qu'une machine puisse représenter.

> **Activité**
>
> Avec une seule feuille de papier (vous pouvez écrire, découper, plier — rien d'autre), fabriquez en binôme une machine capable de représenter les entiers de 0 à 255, sans jamais rien écrire ni effacer pour changer de nombre représenté.
>
> Vous devrez ensuite expliquer : comment un nombre est mémorisé par votre machine, et combien d'éléments de votre machine ont deux états possibles.

Le poids d'une position

Une écriture en base 10 fonctionne par position : dans 61027, le chiffre 6 ne vaut pas 6, il vaut 60000, parce qu'il est en position des dizaines de milliers.

Position	10^4	10^3	10^2	10^1	10^0
Chiffre	6	1	0	2	7
Valeur	60000	1000	0	20	7

C'est exactement le même principe en base 2 : chaque position a un **poids**, qui est une puissance de 2 au lieu d'une puissance de 10.

Position (poids)	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Valeur du poids	128	64	32	16	8	4	2	1
Bit (exemple)	0	1	0	0	1	1	0	1

La valeur en base 10 est la **somme des poids des positions à 1** :

64 + 8 + 4 + 1 = **77**

L'écriture binaire 01001101 représente donc le nombre 77.

> **À retenir**
>
> 1. Deux états (bit) suffisent à coder n'importe quelle information. 2. Chaque position d'une écriture binaire a un poids, qui est une puissance de 2. 3. La valeur en base 10 est la somme des poids des positions valant 1.

> **Attention**
>
> Le nombre de bits disponibles limite le plus grand nombre représentable. Avec n bits, on peut représenter les entiers de 0 à $2^n - 1$ (par exemple, 0 à 255 avec 8 bits — un octet).

Exercices

1. Quelle est l'écriture décimale du nombre d'écriture binaire `10101100` ?
2. Décomposer 57 à l'aide des puissances de 2 pour retrouver son écriture binaire sur un octet.
3. Combien de bits faut-il au minimum pour représenter le nombre 200 ?
4. Vrai ou faux : une écriture binaire de 4 bits permet de coder les entiers de 0 à 16.

Partie 2 — Convertir les entiers : méthode, base 16 et Python

En partie 1, on devinait la décomposition d'un nombre en puissances de 2. Il faut maintenant une **méthode qui marche toujours**, que l'on peut confier à une machine. Pour la comprendre, il faut d'abord savoir suivre ce qu'un programme garde en mémoire à chaque étape.

Suivre l'état des variables

L'**état** d'un programme, à un instant donné, c'est la valeur de chacune de ses variables. Pour comprendre une boucle, on note cet état dans un **tableau de suivi** : une colonne par variable, une ligne par tour, et les valeurs **à la fin** du tour. La première ligne contient l'état **avant** la boucle.

> **À retenir**
>
> Tracer d'abord, exécuter ensuite. Un tableau de suivi rempli sur papier montre *pourquoi* le programme donne son résultat ; l'exécution ne montre que *quel* résultat il donne.

Exercice 2.1 — Une boucle qui divise

Lisez cette fonction **sans l'exécuter**. `n % 2` est le **reste** de la division de `n` par 2 ; `str(…)` transforme un nombre en texte.

```
def ecrire_binaire(n):
    bits = ""
    while n > 0:
        reste = n % 2
        bits = str(reste) + bits
        n = n // 2
    return bits

print(ecrire_binaire(13))
```

1. Complétez sur papier le tableau de suivi de `ecrire_binaire(13)`.

Tour	`reste`	`bits` à la fin du tour	`n` à la fin du tour
avant la boucle	—	`""`	13
1	1	`"1"`	6
2	…	…	…
3	…	…	…
4	…	…	…

2. Pourquoi la boucle s'arrête-t-elle ? Que renvoie la fonction ? Vérifiez avec la partie 1 : 13 = 8 + 4 + 1.
3. Dans `bits = str(reste) + bits`, le nouveau chiffre est placé **devant** ceux déjà obtenus. Que renverrait la fonction avec `bits = bits + str(reste)` ?
4. Que renvoie `ecrire_binaire(0)` ? Est-ce satisfaisant ?

Exercice 2.2 — L'état dans une fonction

Prévoyez l'affichage **sans exécuter**, puis vérifiez.

```
def moitie(n):
    n = n // 2
    return n

n = 26
m = moitie(n)
print(n, m)
m = moitie(m)
print(n, m)
```

1. Qu'affiche le programme ?
2. La fonction modifie `n`. Pourquoi le `n` du programme principal vaut-il toujours 26 ?

> **À retenir**
>
> Un paramètre et les variables créées dans une fonction sont **locaux** : ils naissent à l'appel et disparaissent au `return`. Le programme principal ne récupère que la **valeur renvoyée**, et seulement s'il la range dans une variable.

La méthode des divisions successives

La fonction `ecrire_binaire` applique une méthode que l'on peut faire à la main :

1. diviser le nombre par 2, noter le **reste** (0 ou 1) et garder le **quotient** ;
2. recommencer avec le quotient, jusqu'à obtenir un quotient nul ;
3. lire les restes **du dernier au premier**.

Division	Quotient	Reste
13 ÷ 2	6	**1**
6 ÷ 2	3	**0**
3 ÷ 2	1	**1**
1 ÷ 2	0	**1**

En lisant les restes de bas en haut : 13 s'écrit **1101** en base 2.

Le premier reste dit si le nombre est pair ou impair : c'est le bit de poids 1, celui de **droite**. Chaque division par 2 décale l'écriture d'un cran vers la droite et fait « tomber » le bit suivant. Voilà pourquoi les restes se lisent à l'envers.

> **Repère**
>
> La méthode fonctionne dans n'importe quelle base $b \geq 2$: on divise par b et les restes vont de 0 à $b - 1$.

La base 16

Un octet s'écrit avec huit chiffres en binaire : c'est long et on s'y trompe vite. On utilise donc souvent la **base 16** (hexadécimal). Il faut seize chiffres : 0 à 9, puis les lettres A à F.

Base 10	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
Base 16	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

Les poids sont les puissances de 16 : 1, 16, 256, 4096… Par exemple `2F` vaut $2 \times 16 + 15 = \mathbf{47}$.

Le grand intérêt de la base 16 : $16 = 2^4$, donc **un chiffre hexadécimal correspond exactement à quatre bits** (un quartet). Pour passer du binaire à l'hexadécimal, on découpe l'écriture en paquets de quatre bits **à partir de la droite**, puis on lit chaque paquet avec les poids 8, 4, 2, 1 — comme `lire_quartet` du travail à distance.

Quartet	1011	0110
Valeur	8 + 2 + 1 = 11	4 + 2 = 6
Chiffre hexadécimal	B	6

L'octet `10110110` s'écrit donc **B6** en base 16. Le chemin inverse est aussi direct : chaque chiffre hexadécimal redonne ses quatre bits.

> **Repère**
>
> L'hexadécimal est partout où l'on manipule des octets : couleurs du web (`#FF8800` = trois octets rouge, vert, bleu), adresses MAC des cartes réseau (`3C:52:82:1A:F0:07`), adresses IPv6, contenu brut d'un fichier.

Les entiers en Python

Python sait écrire et lire un entier dans les trois bases. Ce sont **les mêmes nombres** écrits autrement : la machine, elle, stocke toujours du binaire.

```
>>> 0b1101          # écriture binaire
13
>>> 0x2F            # écriture hexadécimale
47
>>> 0b1101 == 13 == 0xD
True
```

Écriture	Résultat	Type du résultat
`bin(13)`	`'0b1101'`	texte (`str`)
`hex(47)`	`'0x2f'`	texte (`str`)
`int("1101", 2)`	`13`	entier (`int`)
`int("2F", 16)`	`47`	entier (`int`)

`bin` et `hex` vont **de l'entier vers le texte** ; `int(texte, base)` va **du texte vers l'entier**. Le préfixe `0b` ou `0x` indique la base de l'écriture.

> **Attention**
>
> `bin(5) + 1` provoque une erreur `TypeError` : `bin(5)` est du texte, pas un nombre. Pour calculer, on travaille sur l'entier ; on convertit en texte seulement pour afficher.

Contrairement à l'octet de la machine des nombres, un entier Python n'est pas limité à 8, 32 ou 64 bits : `2 ** 100` s'affiche en entier. Python réserve autant de mémoire que nécessaire. Dans la plupart des autres langages, la taille d'un entier est fixée à l'avance — nous verrons en partie 3 ce que cela change.

> **À retenir**
>
> 1. Divisions successives par b : les restes, lus du dernier au premier, donnent l'écriture en base b. 2. Un chiffre hexadécimal = quatre bits. 3. En Python, `0b…` et `0x…` écrivent un entier ; `bin` , `hex` produisent du texte ; `int(texte, base)` redonne un entier.

Exercices

Exercice 2.3 — Écrire `ecrire_binaire`

Réécrivez `ecrire_binaire(n)` **sans regarder** l'exercice 2.1. Elle doit aussi renvoyer `"0"` pour `n = 0`. N'utilisez ni `bin` ni `format`.

```
def ecrire_binaire(n):
    ...
```

```
ecrire_binaire(0) == "0"
ecrire_binaire(1) == "1"
ecrire_binaire(13) == "1101"
ecrire_binaire(57) == "111001"
ecrire_binaire(128) == "10000000"
ecrire_binaire(255) == "11111111"
ecrire_binaire(1000) == "1111101000"
```

Exercice 2.4 — À la main

Sur papier, en montrant la méthode :

1. Écrire 45 en base 2 par divisions successives.
2. Écrire l'octet `11010011` en base 16.
3. Écrire `0x3A` en base 2, puis en base 10.
4. La couleur web `#1E90FF` est formée de trois octets (rouge, vert, bleu). Donner chacun en base 10.

Exercice 2.5 — Prévoir Python

Répondez sans exécuter, puis vérifiez dans la console.

Que vaut `int("101", 2) + 1` ?

- [ ] `"1011"`
- [ ] `102`
- [ ] `6`
- [ ] Une erreur

Que produit `bin(10)` ?

- [ ] L'entier `1010`
- [ ] Le texte `'0b1010'`
- [ ] Le texte `'1010'`
- [ ] L'entier `10`

Que se passe-t-il avec `hex(255) + 1` ?

- [ ] On obtient `256`
- [ ] On obtient `'0x100'`
- [ ] Une erreur `TypeError`
- [ ] On obtient `'0xff1'`

Laquelle de ces expressions vaut `True` ?

- [ ] `0b11 == 11`

- [ ] `0xF == 15`
- [ ] `bin(3) == 0b11`
- [ ] `int("F", 2) == 15`

Exercice 2.6 — Pour aller plus loin : écrire en base 16
Adaptez la méthode à la base 16. Un reste va de 0 à 15 ; pour obtenir son chiffre, on utilise le texte `chiffres = "0123456789ABCDEF"` : `chiffres[11]` vaut `"B"`.

```
def ecrire_hexa(n):
    chiffres = "0123456789ABCDEF"
    ...
```

```
ecrire_hexa(0) == "0"
ecrire_hexa(9) == "9"
ecrire_hexa(11) == "B"
ecrire_hexa(47) == "2F"
ecrire_hexa(255) == "FF"
ecrire_hexa(256) == "100"
ecrire_hexa(48879) == "BEEF"
```

Partie 3 — Taille d'une écriture binaire

Dans la machine des nombres, huit bits s'arrêtaient à 255. Python, lui, écrit `2 ** 100` sans broncher. Mais la plupart des langages et des processeurs travaillent avec une taille **fixée à l'avance** : 8, 16, 32 ou 64 bits. Il faut donc savoir prévoir combien de bits un nombre demande, et ce qui arrive quand il n'y en a pas assez.

Combien de valeurs avec *n* bits ?
Chaque bit ajouté double le nombre de configurations possibles : avec n bits, on écrit $\mathbf{2^n}$ **valeurs**, de 0 à $\mathbf{2^n - 1}$.

Bits	Valeurs différentes	Plus grand entier positif	Exemple d'usage
1	2	1	vrai / faux
4	16	15	un chiffre hexadécimal
8	256	255	un octet, une composante de couleur
16	65 536	65 535	numéro de port TCP
32	4 294 967 296	environ 4,3 milliards	adresse IPv4
64	environ $1{,}8 \times 10^{19}$	environ $1{,}8 \times 10^{19}$	entiers d'un processeur actuel

> **Repère**
> $2^{10} = 1024$, à peu près 1000. D'où un calcul mental rapide : $2^{20} \approx$ un million, $2^{30} \approx$ un milliard.

Combien de bits pour écrire un entier ?
Le nombre de bits nécessaire pour écrire un entier $N \geq 1$ est le plus petit n tel que $N \leq 2^n - 1$, autrement dit tel que $N < 2^n$. C'est aussi la **longueur de son écriture binaire**, sans zéros à gauche.

Exemple : 200. On a $2^7 = 128 \leq 200 < 256 = 2^8$, donc 200 demande **8 bits** (`11001000`). Et 256 demande 9 bits : `100000000`.

> **À retenir**
>
> N demande n bits quand $2^{n-1} \leq N < 2^n$: on encadre N entre deux puissances de 2 consécutives.

En Python, deux façons d'obtenir ce nombre de bits :

```
>>> len(bin(200)) - 2      # on retire le préfixe 0b
8
>>> (200).bit_length()
8
```

Exercice 3.1 — Compter les divisions
Lisez cette fonction **sans l'exécuter**.

```
def nb_bits(n):
    compteur = 1
    while n > 1:
        n = n // 2
        compteur = compteur + 1
    return compteur

print(nb_bits(13))
```

1. Complétez le tableau de suivi de `nb_bits(13)`.

Tour	`n` à la fin du tour	`compteur` à la fin du tour
avant la boucle	13	1
1	…	…
2	…	…
3	…	…

2. Que renvoie `nb_bits(13)` ? Est-ce cohérent avec `1101` ?
3. Prévoyez `nb_bits(15)` et `nb_bits(16)` sans tableau complet, puis vérifiez.
4. Pourquoi le compteur commence-t-il à 1 et non à 0 ? Que renvoie `nb_bits(0)` ?

Somme et produit
Combien de bits faut-il pour le **résultat** d'un calcul ? On raisonne sur les plus grandes valeurs possibles.

Opération	Plus grand résultat	Taille du résultat
Somme de deux entiers de n bits	$(2^n - 1) + (2^n - 1) = 2^{n+1} - 2$	au plus **n + 1** bits
Produit d'un entier de n bits par un entier de m bits	$(2^n - 1)(2^m - 1) < 2^{n+m}$	au plus **n + m** bits

Exemples sur 8 bits : 255 + 255 = 510 demande 9 bits ; 255 × 255 = 65 025 demande 16 bits.

Quand les bits manquent : le dépassement
Sur une taille fixe, un résultat trop grand **ne tient pas** : le bit en trop est perdu. Sur 8 bits, 255 + 1 donne `1 00000000` ; seuls les huit bits de droite restent, et le résultat vaut **0**. C'est un **dépassement de capacité** (*overflow*). On compte alors « modulo 256 », comme un compteur kilométrique qui repasse à zéro.

Système	Taille	Ce qui s'est produit
Pac-Man (1980)	compteur de niveau sur 8 bits	au niveau 256, la moitié de l'écran devient illisible
Ariane 5, vol 501 (1996)	entier sur 16 bits	une valeur trop grande pour 16 bits provoque l'arrêt du calculateur et la perte du lanceur
Compteur de vues YouTube (2014)	entier signé sur 32 bits	*Gangnam Style* approche 2 147 483 647 vues : passage à 64 bits
Horloge Unix	entier signé sur 32 bits	le 19 janvier 2038, le compteur de secondes dépasse sa limite

> **Attention**
>
> Python ne déborde pas sur ses entiers, mais les programmes qui échangent avec des fichiers, des réseaux ou des capteurs manipulent des tailles fixes. Vérifier qu'un résultat tient dans la taille prévue fait partie de la **mise au point**.

Exercices

Exercice 3.2 — À la main

1. Combien de bits faut-il pour écrire 100 ? 255 ? 256 ? 1000 ?
2. Combien de bits faut-il pour écrire un million ? Utilisez $2^{10} \approx 1000$.
3. Deux entiers s'écrivent chacun sur 8 bits. Combien de bits au plus pour leur somme ? pour leur produit ?
4. Un entier de 8 bits est multiplié par un entier de 4 bits. Donnez le plus grand produit possible et le nombre de bits nécessaire.

Exercice 3.3 — Une addition sur 8 bits

Écrivez `addition_8_bits(a, b)` qui simule une machine à 8 bits : elle renvoie la somme de `a` et `b` en ne gardant que les huit bits de droite. Indice : garder les huit bits de droite revient à prendre le reste de la division par 256.

```
def addition_8_bits(a, b):
    ...
```

```
addition_8_bits(100, 50) == 150
addition_8_bits(200, 55) == 255
addition_8_bits(255, 1) == 0
addition_8_bits(200, 100) == 44
addition_8_bits(255, 255) == 254
```

Puis écrivez `deborde(a, b)` qui renvoie `True` si la somme ne tient pas sur 8 bits.

```
def deborde(a, b):
    ...
```

```
deborde(100, 50) == False
deborde(200, 55) == False
deborde(255, 1) == True
deborde(128, 128) == True
```

Exercice 3.4 — Vérifier sa compréhension

Combien de bits faut-il pour écrire 64 ?

- [ ] 6

- [ ] 7
- [ ] 8
- [ ] 64

Combien d'adresses IPv4 différentes peut-on écrire sur 32 bits ?

- [ ] 32
- [ ] 4 294 967 295
- [ ] 4 294 967 296
- [ ] 2 147 483 647

Que vaut `(255).bit_length()` ?

- [ ] `8`
- [ ] `255`
- [ ] `9`
- [ ] `'0b11111111'`

Exercice 3.5 — Pour aller plus loin : le compteur de *Gangnam Style*

Le compteur de vues de YouTube utilisait un entier sur 32 bits, et sa valeur maximale était 2 147 483 647.

1. Vérifiez en Python que $2\,147\,483\,647 = 2^{31} - 1$.
2. Avec 32 bits, on pourrait aller jusqu'à $2^{32} - 1$. Proposez une explication à ce bit « manquant » : que faudrait-il pouvoir représenter d'autre qu'un nombre positif ?

Partie 4 — Entiers relatifs et complément à 2

Contenu à venir : représenter un entier négatif sur 8, 16, 32 ou 64 bits.

Corrigés

Première NSI

Représenter les entiers avec deux états, en bases 2, 10 et 16, puis en Python

Représentation des données

Partie 1

1. `10101100` vaut 128 + 32 + 8 + 4, soit **172** en base 10.
2. 57 = 32 + 16 + 8 + 1 ; sur un octet, son écriture est donc **00111001**.
3. 7 bits permettent d'aller jusqu'à 127. Il faut donc **8 bits** pour représenter 200.
4. **Faux.** Avec 4 bits, les valeurs possibles vont de 0 à 15 inclus.

Exercice 2.1

Tour	`reste`	`bits` à la fin du tour	`n` à la fin du tour
avant la boucle	—	`""`	13
1	1	`"1"`	6
2	0	`"01"`	3
3	1	`"101"`	1
4	1	`"1101"`	0

1. Voir le tableau.
2. Après le 4e tour, `n` vaut 0 : la condition `n > 0` est fausse. La fonction renvoie `"1101"`, et 8 + 4 + 1 = 13.
3. Les restes seraient dans l'ordre de leur calcul : `"1011"`, qui vaut 11. L'écriture serait à l'envers.
4. Pour 0, on n'entre jamais dans la boucle : la fonction renvoie le texte vide `""`. Il faudrait renvoyer `"0"` : c'est le cas à traiter à part dans l'exercice 2.3.

Exercice 2.2

1. Le programme affiche `26 13`, puis `26 6`.
2. Le `n` de la fonction est une autre variable, locale, qui reçoit une copie de la valeur 26 à l'appel. La modifier ne change pas le `n` du programme principal. Seule la valeur renvoyée sort de la fonction, ici rangée dans `m`.

Exercice 2.3

Indice : commencer par traiter `n == 0` avec un `if` et un `return`, puis reprendre la boucle de l'exercice 2.1.

```
def ecrire_binaire(n):
    if n == 0:
        return "0"
    bits = ""
    while n > 0:
        bits = str(n % 2) + bits
        n = n // 2
    return bits
```

Exercice 2.4

1. Restes successifs de 45 : 1, 0, 1, 1, 0, 1. Lus du dernier au premier : **101101** (32 + 8 + 4 + 1 = 45).

2. `1101` = 13 = D et `0011` = 3 : **D3**.
3. `3` = `0011` et `A` = `1010` : **00111010** ; 3 × 16 + 10 = **58**.
4. `1E` = 1 × 16 + 14 = **30** ; `90` = 9 × 16 = **144** ; `FF` = **255**.

Exercice 2.5

Réponses : `6`, le texte `'0b1010'`, une erreur `TypeError`, `0xF == 15`.

Exercice 2.6

Indice : même structure que `ecrire_binaire`, avec `// 16` et `% 16` ; le reste sert de position dans `chiffres`.

```
def ecrire_hexa(n):
    chiffres = "0123456789ABCDEF"
    if n == 0:
        return "0"
    texte = ""
    while n > 0:
        texte = chiffres[n % 16] + texte
        n = n // 16
    return texte
```

Exercice 3.1

Tour	`n` à la fin du tour	`compteur` à la fin du tour
avant la boucle	13	1
1	6	2
2	3	3
3	1	4

1. Voir le tableau : après le 3e tour, `n` vaut 1 et la boucle s'arrête.
2. `nb_bits(13)` renvoie 4 : `1101` a bien quatre chiffres.
3. `nb_bits(15)` vaut 4 (`1111`), `nb_bits(16)` vaut 5 (`10000`) : 16 = 2^4 est le premier nombre qui demande un bit de plus.
4. Un entier supérieur ou égal à 1 demande au moins un bit : on compte celui de gauche dès le départ, puis un bit par division. `nb_bits(0)` renvoie 1, ce qui est correct : on écrit 0 avec un bit.

Exercice 3.2

1. 100 : 64 ≤ 100 < 128, **7 bits**. 255 : **8 bits**. 256 = 2^8 : **9 bits**. 1000 : 512 ≤ 1000 < 1024, **10 bits**.
2. 2^{20} ≈ 1 000 000 mais 2^{20} = 1 048 576 > 1 000 000 et 2^{19} = 524 288 ≤ 1 000 000 : **20 bits**.
3. Somme : au plus **9 bits** (255 + 255 = 510). Produit : au plus **16 bits** (255 × 255 = 65 025).
4. 255 × 15 = **3825** ; 2048 ≤ 3825 < 4096 : **12 bits** = 8 + 4.

Exercice 3.3

```
def addition_8_bits(a, b):
    return (a + b) % 256

def deborde(a, b):
    return a + b > 255
```

`(a + b) % 256` garde le reste de la division par 256 = 2^8, c'est-à-dire les huit bits de droite.

Exercice 3.4

Réponses : 7 bits ; 4 294 967 296 adresses ; `8`.

Exercice 3.5

1. `2 ** 31 - 1 == 2147483647` vaut `True`.
2. Un bit sert à indiquer le **signe** : le compteur était un entier pouvant être négatif. Il reste 31 bits pour la valeur. La partie 4 montre comment on code les entiers négatifs.